Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No:92301733057**

Aim: To study different word embedding for representation of textual data in vectorized form

IDE: Google Colab

Theory:

Word Embedding is an approach for representing words and documents. Word Embedding or Word Vector is a numeric vector input that represents a word in a lower-dimensional space. It allows words with similar meanings to have a similar representation.

Word Embeddings are a method of extracting features out of text so that we can input those features into a machine learning model to work with text data. They try to preserve syntactical and semantic information. The methods such as Bag of Words (BOW), CountVectorizer and TFIDF rely on the word count in a sentence but do not save any syntactical or semantic information. In these algorithms, the size of the vector is the number of elements in the vocabulary. We can get a sparse matrix if most of the elements are zero. Large input vectors will mean a huge number of weights which will result in high computation required for training. Word Embeddings give a solution to these problems.

Need for Word Embedding?

- To reduce dimensionality
- To use a word to predict the words around it.
- Inter-word semantics must be captured.

How are Word Embeddings used?

- They are used as input to machine learning models. Take the words Give their numeric representation Use in training or inference.
- To represent or visualize any underlying patterns of usage in the corpus that was used to train them.

1. Bag of Words

Bag of Words (BOW) is an algorithm that counts how many times a word appears in a document. Those word counts allow us to compare documents and gauge their similarities for applications like search, document classification, and topic modeling.

2. Tf-Idf

Tf-Idf is shorthand for term frequency-inverse document frequency. So, two things: term frequency and inverse document frequency. Term frequency (TF) is basically the output of the BoW model. For a specific document, it determines how important a word is by looking at how frequently it appears in the document. Term frequency measures the importance of the word. If a word appears a lot of times, then the word must be important. For example, if our document is “I am a cat lover. I have a cat named Steve. I feed a cat outside my room regularly,” we see that the words with the highest frequency are I, a, and cat. This agrees with our intuition that high term frequency = higher importance.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No:92301733057**

TF(t) = (Number of times term t appears in a document) / (Total number of terms in the document)

IDF used to calculate the weight of rare words across all documents. The words that occur rarely in the corpus have a high IDF score. However, it is known that certain terms, such as "I", "a" may appear a lot of times but have little importance. Thus we need to weigh down the frequent terms while scaling up the rare ones

IDF(t) = log_e(Total number of documents / Number of documents with term t in it)

Pre Lab Exercise:

1. **Perform BoW vectorization of the corpus**

BoW: Converts text into vectors based on word count

Program(Code):

```
from sklearn.feature_extraction.text import CountVectorizer

# Sample corpus
corpus = [
    "I love machine learning",
    "Machine learning is amazing",
    "I love coding"
]

# Create BoW model
vectorizer = CountVectorizer()

# Transform text to BoW vectors
X = vectorizer.fit_transform(corpus)

# Convert to array
print("Vocabulary:", vectorizer.get_feature_names_out())
print("\nBoW Matrix:\n", X.toarray())
```

Results:

```
Vocabulary: ['amazing' 'coding' 'is' 'learning' 'love' 'machine']

BoW Matrix:
 [[0 0 0 1 1 1]
 [1 0 1 1 0 1]
 [0 1 0 0 1 0]]
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No:92301733057**

3. Perform TF vectorization of the corpus

Program(Code):

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Same corpus
corpus = [
    "I love machine learning",
    "Machine learning is amazing",
    "I love coding"
]

# Create TF-IDF model (gives TF values also)
tf_vectorizer = TfidfVectorizer(use_idf=False)

# Transform text
X_tf = tf_vectorizer.fit_transform(corpus)

# Output
print("Vocabulary:", tf_vectorizer.get_feature_names_out())
print("\nTF Matrix:\n", X_tf.toarray())
```

Results:

```
Vocabulary: ['amazing' 'coding' 'is' 'learning' 'love' 'machine']

TF Matrix:
 [[0.         0.         0.         0.57735027 0.57735027 0.57735027]
 [0.5        0.         0.5        0.5        0.         0.5       ]
 [0.         0.70710678 0.         0.         0.70710678 0.        ]]
```

Program (Code):

To be attached with

1. Perform the TF-IDF vectorization of the corpus

```
sentences = [
  "India is a country known for its rich culture and diversity.",
  "The capital of India is New Delhi.",
  "India is famous for festivals, food, and historical places.",

  "Cricket is one of the most popular games in India.",
  "Playing games helps improve teamwork and concentration.",
```

```
  "Technology makes our daily life faster and more connected. ",
  "AI subject makes our life smarter."
]
#IMPORTING BASIC LIBRARIES

import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer #TF-BoW
from sklearn.feature_extraction.text import TfidfVectorizer #TF-BoW
from sklearn.metrics.pairwise import cosine_similarity

#TF-BoW Embedding
tf_vectorizer=CountVectorizer()
tf_emb=tf_vectorizer.fit_transform(sentences)

#similarity matrix using cosine_similarity
tf_sim=cosine_similarity(tf_emb)


#TF-IDF Embedding
tfidf_vectorizer=TfidfVectorizer()
tfidf_emb=tfidf_vectorizer.fit_transform(sentences)

tfidf_sim=cosine_similarity(tfidf_emb)

tfidf_sim

#Recommend the most matching/ similar sentence
def doc_sim(tfidf_sim,sentences):
  for i in range(len(sentences)):
    sim=tfidf_sim[i].copy()
    sim[i]=-1
    top_index=np.argmax(sim)
    print(sentences[i],"->",sentences[top_index])

doc_sim(tfidf_sim,sentences)
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No:92301733057**

Results:

```
India is a country known for its rich culture and diversity. -> India is famous for festivals,
food, and historical places.
The capital of India is New Delhi. -> Cricket is one of the most popular games in India.
India is famous for festivals, food, and historical places. -> India is a country known for its
rich culture and diversity.
Cricket is one of the most popular games in India. -> The capital of India is New Delhi.
Playing games helps improve teamwork and concentration. -> Cricket is one of the most popular
games in India.
Technology makes our daily life faster and more connected.  -> AI subject makes our life
smarter.
AI subject makes our life smarter. -> Technology makes our daily life faster and more
connected.
```

Observation:

Word embedding techniques such as Bag of Words (BoW), Term Frequency (TF), and TF-IDF convert text into numerical representations. BoW focuses on the frequency of words without considering context, TF normalizes word frequency within a document, and TF-IDF assigns higher importance to rare words while reducing the influence of commonly occurring ones. These approaches make it easier to compare documents and measure similarity between them.

Post Lab Exercise:

Take three documents and find similarity using

Program (Code):

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Sample documents
docs = [
    "I love machine learning",
    "Machine learning is amazing",
    "I enjoy coding and learning"
]
```

a. **BoW vectorization**

```
#BOW
bow = CountVectorizer()
bow_matrix = bow.fit_transform(docs)
```

```
bow_sim = cosine_similarity(bow_matrix)
print("BoW Similarity:\n", bow_sim)
```

```
BoW Similarity:
 [[1.         0.57735027 0.28867513]
 [0.57735027 1.         0.25      ]
 [0.28867513 0.25       1.        ]]
```

b. **TF Vectorization**

```
tf = TfidfVectorizer(use_idf=False)
tf_matrix = tf.fit_transform(docs)

tf_sim = cosine_similarity(tf_matrix)
print("\nTF Similarity:\n", tf_sim)
```

```
TF Similarity:
 [[1.         0.57735027 0.28867513]
 [0.57735027 1.         0.25      ]
 [0.28867513 0.25       1.        ]]
```

c. **TF-IDF Vectorization**

```
#  TF-IDF
tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(docs)

tfidf_sim = cosine_similarity(tfidf_matrix)
print("\nTF-IDF Similarity:\n", tfidf_sim)
```

```
TF-IDF Similarity:
 [[1.         0.39038344 0.13730861]
 [0.39038344 1.         0.11141304]
 [0.13730861 0.11141304 1.        ]]
```